

¿Por qué HTTP devuelve Observable y no la data directa?

Porque la respuesta del servidor no llega inmediatamente. Angular devuelve un Observable para representar datos que llegarán en el futuro, sin congelar la aplicación. Además, los Observables se pueden cancelar con `.unsubscribe()` y permiten usar operadores como `map()` o `catchError()`.

¿Qué pasa si nunca se hace `.subscribe()`?

No pasa nada. El Observable no ejecuta la petición hasta que alguien se suscribe con `.subscribe()`. Si no hay suscripción, la petición HTTP nunca sale.

¿Ventaja de `providedIn: 'root'` vs `new CityService()`?

Con `providedIn: 'root'`, Angular crea y administra una sola instancia del servicio para toda la aplicación. Así puede inyectarle dependencias como `HttpClient`. En cambio, con `new CityService()`, Angular no controla el servicio y no puede inyectarle esas dependencias.

¿Cómo facilita las pruebas unitarias?

Permite reemplazar el servicio real por un mock. Así las pruebas no dependen del backend y se pueden simular respuestas HTTP, por ejemplo con `HttpClientTestingModule`.

¿Por qué usar `HttpInterceptor` y no `try/catch` en cada componente?

Porque evita repetir el mismo manejo de errores en muchos componentes. El interceptor centraliza esa lógica y se aplica automáticamente a todas las peticiones HTTP.

Usos adicionales:

Agregar automáticamente el token: `Authorization: Bearer ...`

Mostrar y ocultar un spinner global mientras hay peticiones activas.

¿Por qué usar `@Input()` en vez de otro `GET /api/cities/{id}`? Porque el componente padre ya tiene la ciudad cargada. Con `@Input()` se pasa directamente al hijo, sin hacer otra petición innecesaria.

¿Ventaja de retornar `country` como objeto anidado?

Evita hacer una petición extra por cada ciudad para buscar el país. Si `/api/cities` ya devuelve la ciudad con su `country`, la app es más rápida y eficiente.